Question 1: Programming: Write a Program - Stacks

Given a string s containing the characters '(' and ')', determine if the input string is valid.

An input string is valid if:

- Open parentheses must be closed in the correct order.
- Every close parenthesis has a corresponding open parenthesis that comes before it.

Example 1:

Input: s = "()"
Output: True

Example 2:

Input: s = "())("
Output: False

Example 3:

Input: s = ")()()("
Output: False

Example 4:

Input: s = "(())()"
Output: True

Example 5:

Input: s = “()()(“
Output: False

Solution:

```
def is_valid(s):
    stack = []
    for c in s:
        if c == '(':
            stack.append(c)
        elif c == ')':
            if not stack:      # no matching '(' beforehand, return False early
                return False
            stack.pop()        # match found, pop from stack
        else:
            return False       # invalid character
    return len(stack) == 0     # all open parentheses should be closed
```

Question 2: Programming: Write a Program - Linked Lists

We want to write code to partition a linked list around a value x, such that all nodes less than x come before all nodes greater than or equal to x. If x is contained within the list, the values of x only need to be after the elements less than x (see below). The partition element x can appear anywhere in the "right partition"; it does not need to appear between the left and right partitions. Either return a new list or modify the existing.

Example
Input: 3 -> 5 -> 8 -> 5 -> 10 -> 2 -> 1 [partition = 5]
Output: 3 -> 1 -> 2 -> 10 -> 5 -> 5 -> 8

Complete the code:

```
class Node:
   """A simple node for a singly linked list."""
   def __init__(self, data=0, next_node=None):
       self.data = data
       self.next = next_node

class LinkedList:
   """A simple linked list class for demonstration purposes."""
   def __init__(self):
       self.head = None

   def append(self, data):
       """Appends a new node with the given data to the end of the list."""
       if not self.head:
           self.head = Node(data)
           return
       current = self.head
       while current.next:
           current = current.next
       current.next = Node(data)

   def print_list(self):
       """Prints the elements of the list."""
       nodes = []
       current = self.head
       while current:
           nodes.append(str(current.data))
           current = current.next
       print(" -> ".join(nodes))
```

```
def partition(node, x):
    """
    Partitions a linked list around a value x, such that all nodes less than x
    come before all nodes greater than or equal to x.


    Args:
  node: The head node of the linked list.
        x: The partition value.


    Returns:
        The head of the new partitioned linked list.
    """
    before_start = None
    before_end = None
    after_start = None
    after_end = None


    # Partition the list into two: one for elements <x and one for elements >= x
    while node is not None:
        next_node = node.next
        node.next = None  # Disconnect the node from the original list


        # COMPLETE THE LOOP BY INSERTING THE NODE INTO THE END OF EITHER THE ‘before’
LIST OR THE ‘after’ LIST

        node = next_node

    if before_start is None:
        return after_start

    # Merge the 'before' and 'after' lists

    return before_start
```

Solution:

```
def partition(node, x):
    before_start = None
    before_end = None
    after_start = None
    after_end = None
```

```
# Partition the list into two: one for elements <x and one for elements >= x
while node is not None:
    next_node = node.next
    node.next = None  # Disconnect the node from the original list

    if node.data < x:
        # Insert node into the end of the 'before' list
        if before_start is None:
            before_start = node
            before_end = before_start
        else:
            before_end.next = node
            before_end = node
    else:
        # Insert node into the end of the 'after' list
        if after_start is None:
            after_start = node
            after_end = after_start
        else:
            after_end.next = node
            after_end = node

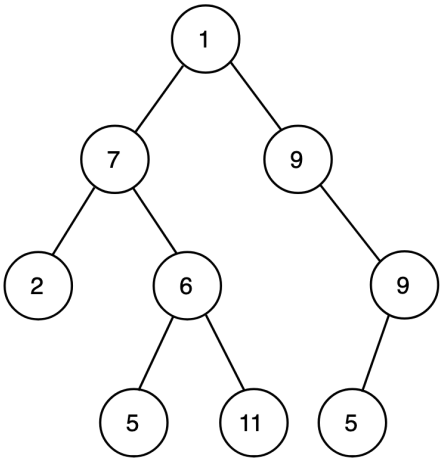
    node = next_node

if before_start is None:
    return after_start

# Merge the 'before' and 'after' lists
before_end.next = after_start
return before_start
```

Question 3: Binary Trees

3a: Trace the following code segment. The input is the tree in the picture below. What is the output? What sort of traversal is this?

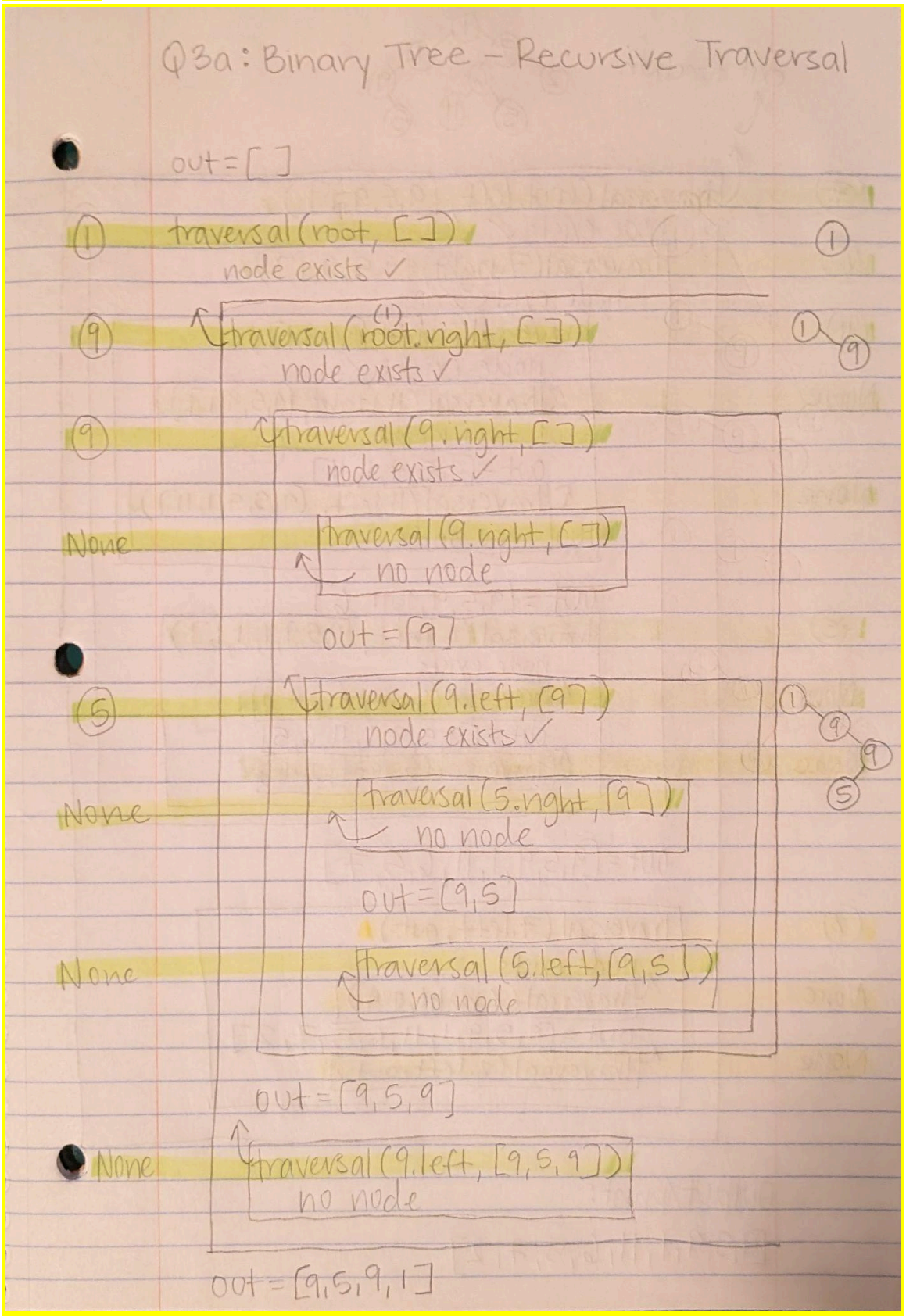


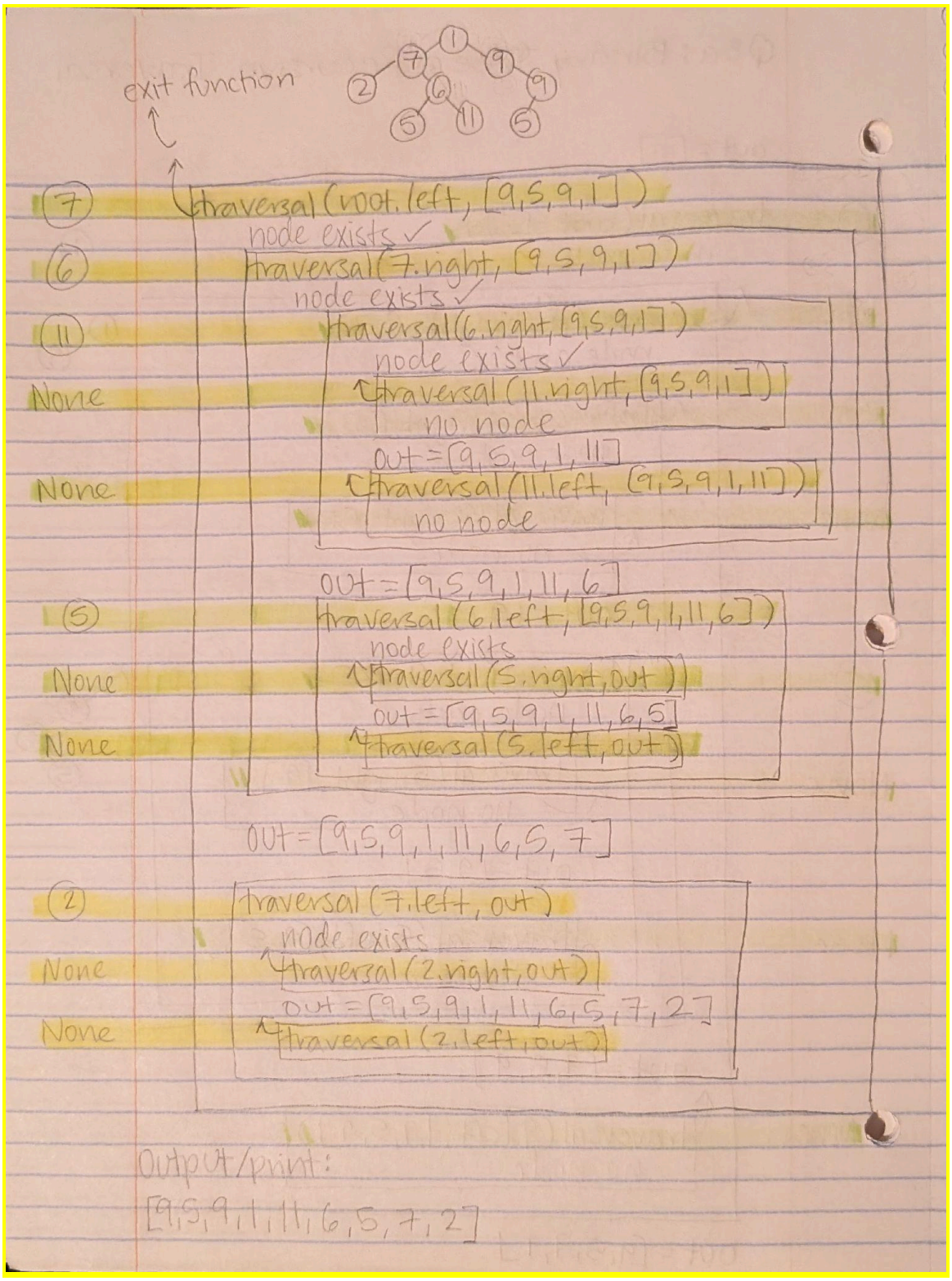

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def traversal(node, out):
    if not node:
        return
    traversal(node.right, out)
    out.append(node.val)
    traversal(node.left, out)

out = []
traversal(root, out)
print(out)
```

Solution:


Q3a: Binary Tree - Recursive Traversal
out = [ ]
1 traversal(root, [ ])
node exists ✓
9 traversal(root.right, [ ])
node exists ✓
9 traversal(9.right, [ ])
node exists ✓
None traversal(9.right, [ ])
no node
out = [9]
5 traversal(9.left, [9])
node exists ✓
None traversal(5.right, [9])
no node
out = [9,5]
None traversal(5.left, [9,5])
no node
out = [9,5,9]
None traversal(9.left, [9,5,9])
no node
out = [9,5,9,1]



Output: [9, 5, 9, 1, 11, 6, 5, 7, 2]
This is a **reversed** In-Order Traversal.

3b: How would the code change if we were doing in-order, pre-order, and post-order traversal? What would the output be for in-order, pre-order, and post-order traversal?

Solution:

- In-Order Traversal: (also ascending order!)
  - Code:
    ```
    traversal(node.left, out)
    out.append(node.val)
    traversal(node.right, out)
    ```
  - Output:
    [2, 7, 5, 6, 11, 1, 9, 5, 9]
- Pre-Order Traversal:
  - Code:
    ```
    out.append(node.val)
    traversal(node.left, out)
    traversal(node.right, out)
    ```
  - Output:
    [1, 7, 2, 6, 5, 11, 9, 9, 5]
- Post-Order Traversal:
  - Code:
    ```
    traversal(node.left, out)
    traversal(node.right, out)
    out.append(node.val)
    ```
  - Output:
    [2, 5, 11, 6, 7, 5, 9, 9, 1]

3c: Binary Search Trees:

3.c.1: What does the following binary search tree look like when we insert 90?

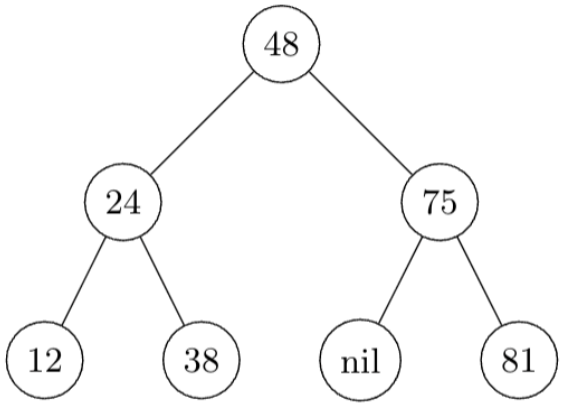


Figure: Insert 90 in the above BST!

Solution:

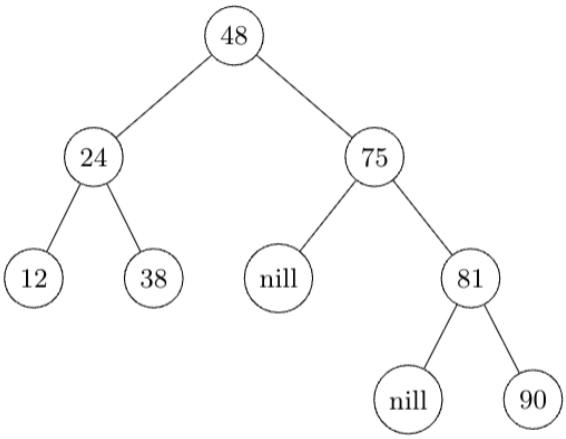


Figure: BST after inserting 90

3.c.2: What does the following binary search tree look like when we delete 400?

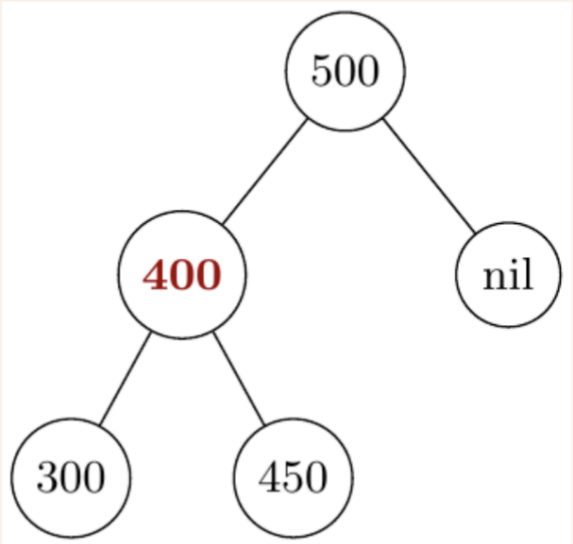


Figure: Remove 400 in the above BST!

Solution:

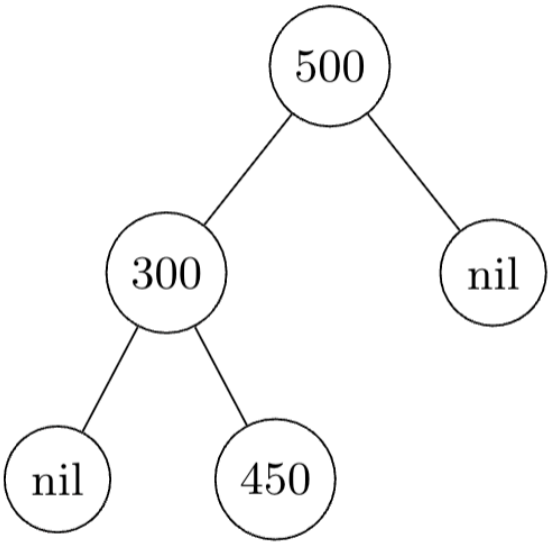


Figure: Resulting BST

3.c.4: What does the following binary search tree look like when we delete 600?

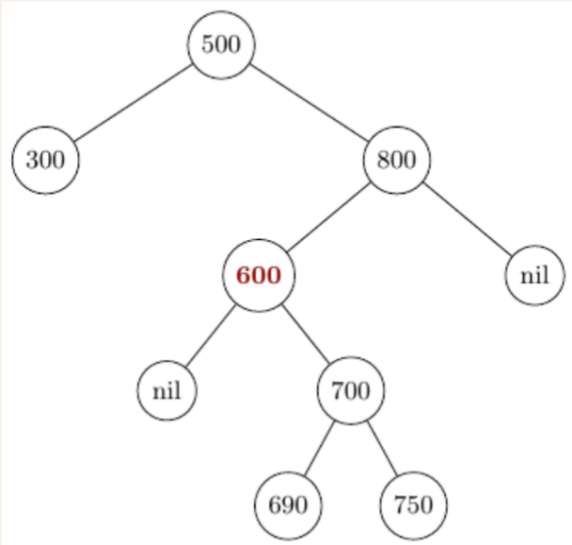


Figure: Remove 600 in the above BST!

Solution:

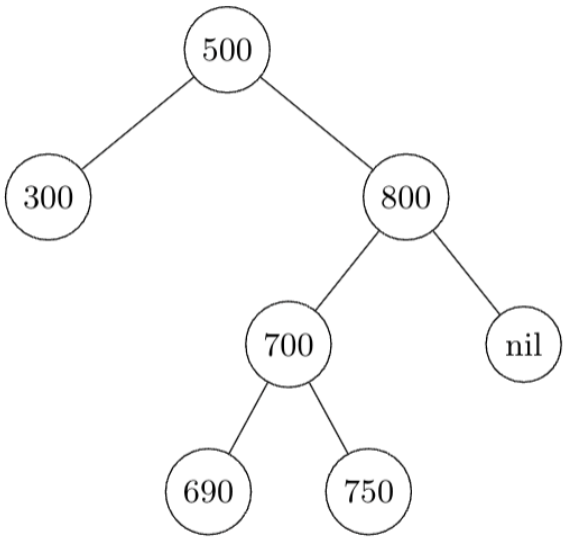


Figure: Resulting BST

Question 4: Code tracing: subset sums

```
def subset_sum(nums, target, index=0, curr_subset=None, curr_sum=0,
result=None):

    if curr_subset is None:
        curr_subset = []
    if result is None:
        result = []


    if curr_sum > target:
        return result


    if curr_sum == target:
        result.append(curr_subset.copy())
        return result


    if index == len(nums):
        return result


    curr_subset.append(nums[index])
    subset_sum(nums, target, index + 1, curr_subset, curr_sum +
nums[index], result)
    curr_subset.pop()  # backtrack


    subset_sum(nums, target, index + 1, curr_subset, curr_sum, result)


nums = [2, 3, 5]
target_sum = 5

solutions = subset_sum(nums, target_sum)
print("Subsets that sum to", target_sum, ":", solutions)

    return result
```

Solution:

→ Concept: Recursive Backtracking

PSS2 Code-Tracing: Subset Sums

variables	values
nums	[2,3,5]
target_sum	5
solutions	subset_sum(nums, target_sum)

subset_sum(nums=[2,3,5], target=5, index=0, curr_subset=None, curr_sum=0, result=None)

curr_subset is None & result is None
curr_subset = [ ]
result = [ ]

0 > 5 → False
0 == 5 → False
0 == 3 → False

curr_subset = [2] ← nums[0]

subset_sum([2,3,5], 5, 1, [2], 2, [ ]) (0+1, 0+)

☆ first 2 if statements are never entered from this point on (curr_subset & result ≠ None) true/

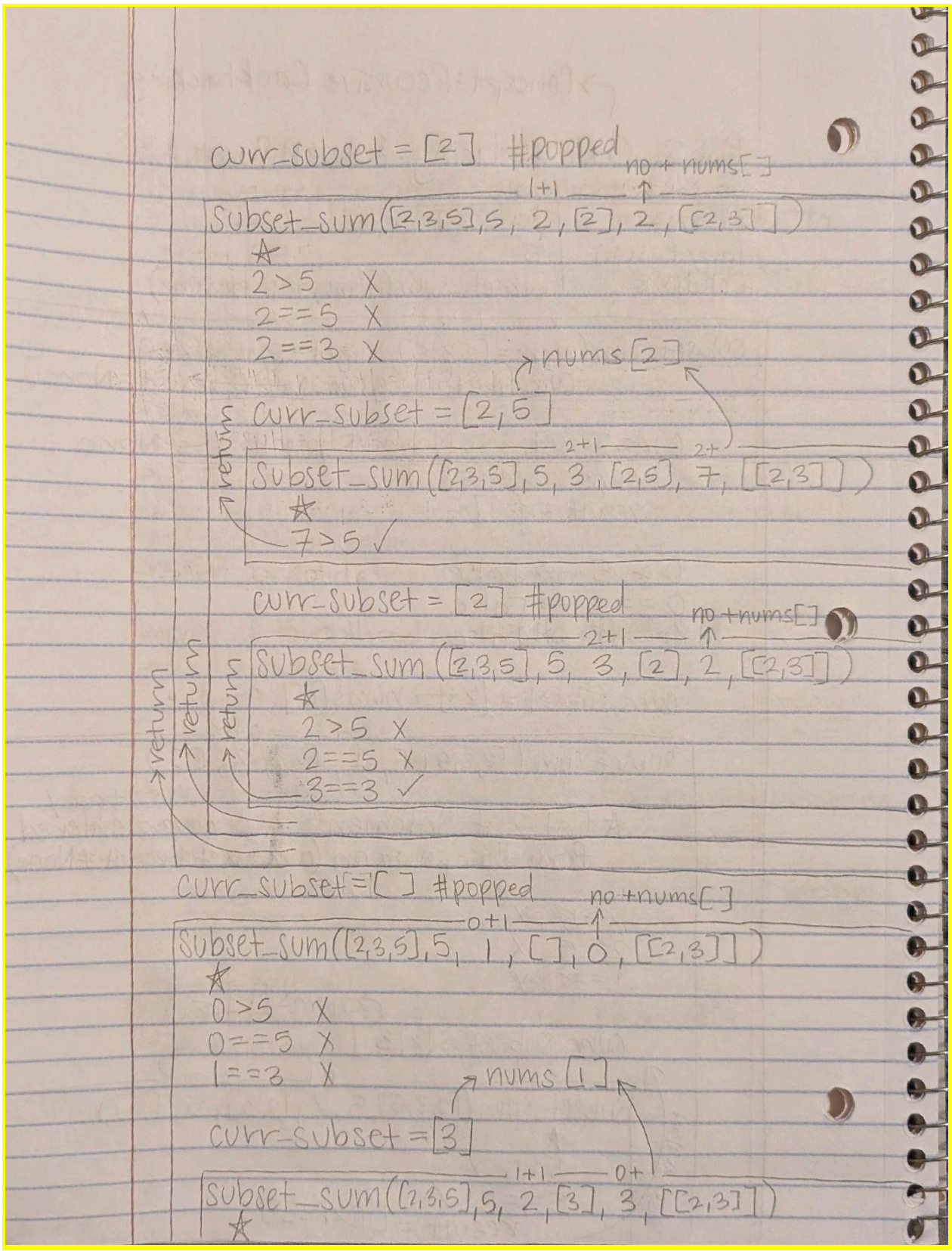
2 > 5 X
2 == 5 X
1 == 3 X

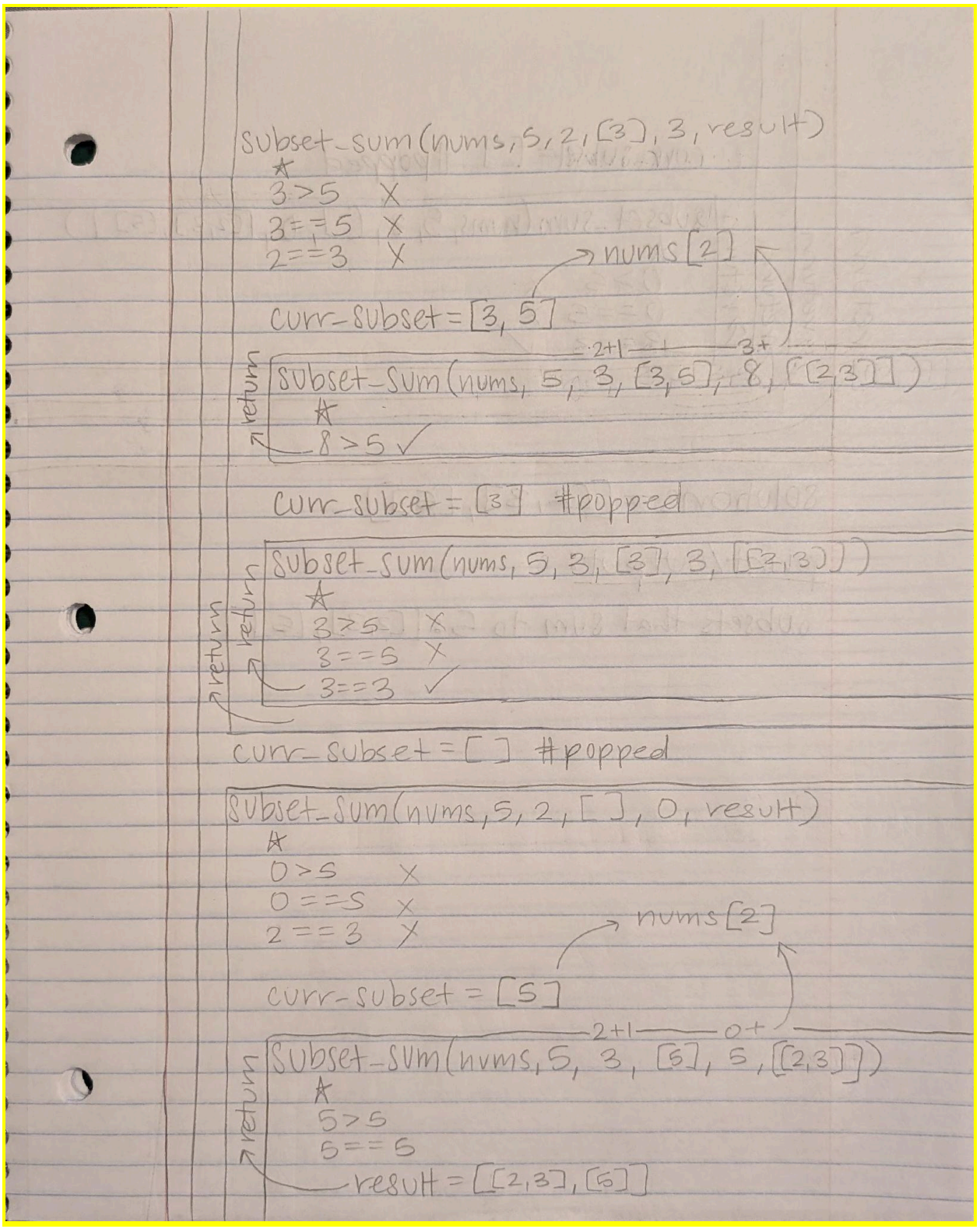
curr_subset = [2,3] ← nums[1]

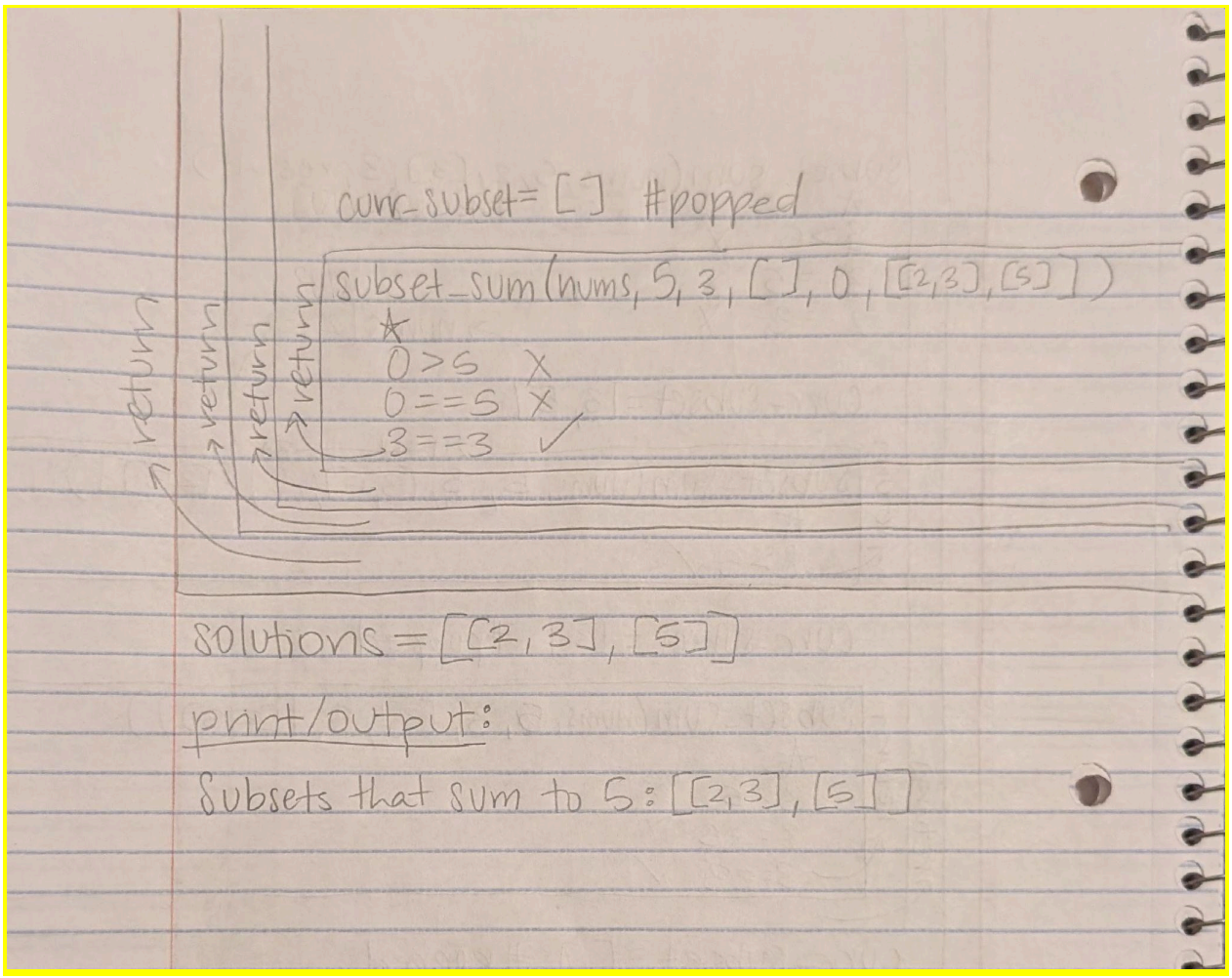
subset_sum([2,3,5], 5, 2, [2,3], 5, [ ]) (1+1, 2+)

☆
5 > 5 X
5 == 5 ✓
result = [[2,3]]

return


curr_subset = [2] #popped
no + nums[ ]
1+1
Subset_sum([2,3,5],5, 2, [2], 2, [[2,3]])
2 > 5 X
2 == 5 X
2 == 3 X
nums [2]
curr_subset = [2,5]
2+1
2+
return
Subset_sum([2,3,5],5, 3, [2,5], 7, [[2,3]])
7 > 5 ✓
curr_subset = [2] #popped
no + nums[ ]
2+1
Subset_sum([2,3,5],5, 3, [2], 2, [[2,3]])
2 > 5 X
2 == 5 X
3 == 3 ✓
return
return
return
curr_subset = [ ] #popped
no + nums[ ]
0+1
Subset_sum([2,3,5],5, 1, [ ], 0, [[2,3]])
0 > 5 X
0 == 5 X
1 == 3 X
nums [1]
curr_subset = [3]
1+1
0+
Subset_sum([2,3,5],5, 2, [3], 3, [[2,3]])

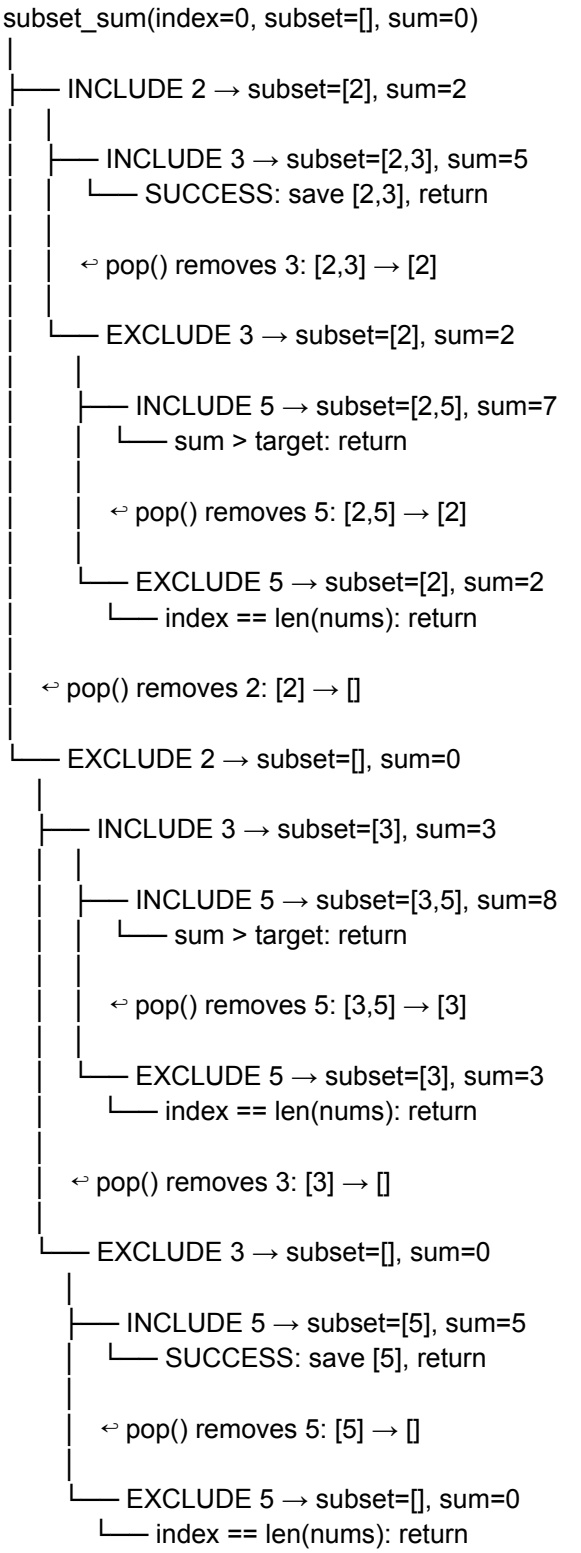

subset_sum(nums, 5, 2, [3], 3, result)
3>5 X
3==5 X
2==3 X
nums[2]
curr_subset = [3, 5]
2+1
3+
return
subset_sum(nums, 5, 3, [3,5], 8, [[2,3]])
8>5 ✓
curr_subset = [3] #popped
subset_sum(nums, 5, 3, [3], 3, [[2,3]])
3>5 X
3==5 X
3==3 ✓
return
return
curr_subset = [] #popped
subset_sum(nums, 5, 2, [], 0, result)
0>5 X
0==5 X
2==3 X
nums[2]
curr_subset = [5]
2+1
0+
return
subset_sum(nums, 5, 3, [5], 5, [[2,3]])
5>5
5==5
result = [[2,3], [5]]



Output

Subsets that sum to 5 : [[2, 3], [5]]

Recursive Backtracking Tree:



Question 5: Programming: Recursive Binary Search Tree implementation

A **Binary Search Tree (BST)** is a data structure that allows efficient insertion and search.

In a BST each node contains a value and at most two children:

1. **Left child** contains values **less than** the parent node.
2. **Right child** contains values **greater than** the parent node.

Duplicates: you can store them in a consistent way (e.g., always on the right).

Below is a partially implemented Python class for BST. You are asked to complete the class definition for Node class' __init__() function, BST class' insert(), and search() functions for BinarySearchTree.

Tasks:

1. Node class:
    a. Store the value, left and right children are initialized correctly.
2. BST class:
    a. Implement insert() using recursion: this function returns the inserted Node object.
        i. Recurse to insert a new Node value based on BST property specified above. Capture the empty case.
    b. Implement search() using recursion: this function determines if a value exists and returns True if the element is found.
        i. Recurse to search for a Node value based on BST property specified above to determine if the Node value exists or not.

Example usage:

```
bst = BinarySearchTree()
bst.insert(10)
bst.insert(5)
bst.insert(15)

print(bst.search(10))  # True
print(bst.search(5))   # True
print(bst.search(15))  # True
print(bst.search(20))  # False
```

```
class Node:
    def __init__(self, value):
        """
        Initialize a BST node with appropriate information.

        Args:
            value (int): The value to store in the node.
        """
    # TODO: add appropriate information

# Binary Search Tree class
class BinarySearchTree:
    def __init__(self):

        self.root = None

    def insert(self, key):

        self.root = self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        """
        Recursive helper for insert.

        """
        # TODO: implement recursion

    def search(self, key):

        return self._search_recursive(self.root, key)

    def _search_recursive(self, node, key):
        """
        Recursive helper for search.

        """
        # TODO: implement recursion
```

Solution:

```
class BSTNode:
    def __init__(self, value):
        """Initialize a BST node."""
```

```
        self.value = value
        self.left = None
        self.right = None


class BinarySearchTree:
    def __init__(self):
        """Initialize an empty BST."""
        self.root = None

    def insert(self, key):
        """Insert a key into the BST."""
        self.root = self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        if node is None:
            return BSTNode(key)

        if key < node.value:
            node.left = self._insert_recursive(node.left, key)
        elif key >= node.value:
            node.right = self._insert_recursive(node.right, key)
        return node

    def search(self, key):
        """Search for a key in the BST."""
        return self._search_recursive(self.root, key)

    def _search_recursive(self, node, key):
        if node is None:
            return False

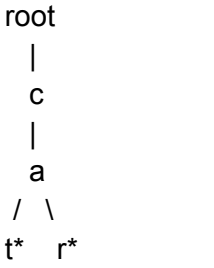
        if key < node.value:
            return self._search_recursive(node.left, key)
        elif key > node.value:
            return self._search_recursive(node.right, key)
        else:
            return True  # key found
```

Question 6: Programming: Recursive Tries (tree)

A Trie is a tree-like data structure used to store sequences (strings or numbers as strings).

- Each node represents a single element of the sequence (character or number).
- Each node may have multiple children, typically stored in a dictionary.
- A flag (e.g., is_end) indicates if the node represents the end of a valid key.
- Insertion and search works based on this property recursively.
- Deletion must carefully remove keys without affecting other keys sharing the same prefix recursively.

Example: insert cat and car in a tree



* indicates is_end = True.

Task

1. Implement a Node class (TrieNode):
    a. Stores a dictionary of children.
    b. Boolean flag is_end to indicate a key ends here.
2. Implement a Trie class:
    a. Contains a reference to the root node.
    b. Supports recursive insert: Adds a key to the Trie, creating nodes as needed.
    c. Supports recursive search: Checks whether a key exists in the Trie.
    d. Supports recursive delete: Check for the existence of the key. If the key exists unmark is_end. Remove any nodes that are no longer needed (no children and is_end is False).Must preserve other keys that share the same prefix.
3. Keys can be strings or numbers (stored as strings for sequence traversal).

Note that keys can be strings or numbers (stored as strings for sequence traversal).

```
class TrieNode:
    def __init__(self):

        # TODO: add appropriate information


class Trie:
    def __init__(self):
```

```
        self.root = TrieNode()

    def insert(self, key):

        self._insert_recursive(self.root, key, 0)

    def _insert_recursive(self, node, key, index):
        """
        Recursive helper for insert.

        """
        # TODO: implement recursion

    def search(self, key):

        return self._search_recursive(self.root, key, 0)

    def _search_recursive(self, node, key, index):
        """
        Recursive helper for search.

        """
        # TODO: implement recursion

    def delete(self, key):

        self._delete_recursive(self.root, key, 0)

    def _delete_recursive(self, node, key, index):
        """
        Recursive helper for delete.

        """
        # TODO: implement recursion
```

Example Usage:

```
trie = Trie()
trie.insert("cat")
trie.insert("car")

print(trie.search("cat"))  # True
print(trie.search("cap"))  # False
print(trie.search("dog"))  # True
```

```
print(trie.search("do"))   # False
```

Solution:

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False


class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        self._insert_recursive(self.root, word, 0)

    def _insert_recursive(self, node, word, index):
        if index == len(word):
            node.is_end = True
            return

        char = word[index]
        if char not in node.children:
            node.children[char] = TrieNode()

        self._insert_recursive(node.children[char], word, index + 1)

    def search(self, word):
        return self._search_recursive(self.root, word, 0)

    def _search_recursive(self, node, word, index):
        if index == len(word):
            return node.is_end

        char = word[index]
        if char not in node.children:
            return False

        return self._search_recursive(node.children[char], word, index + 1)

  def delete(self, key):
        """Delete a key recursively using helper."""
```

```
        self._delete_recursive(self.root, key, 0)

    def _delete_recursive(self, node, key, index):
        """
        Recursive helper for delete.
        Returns True if the current node can be deleted (no children & not
end).
        """
        if index == len(key):
            # Base case: reached end of key
            if not node.is_end:
                return False  # key not found
            node.is_end = False
            # If node has no children, it can be deleted
            return len(node.children) == 0

        char = key[index]
        if char not in node.children:
            return False  # key not found

        should_delete_child = self._delete_recursive(node.children[char],
key, index + 1)

        # If child can be deleted, remove it
        if should_delete_child:
            del node.children[char]

        # Return True if current node can also be deleted
        return not node.is_end and len(node.children) == 0
```

Question 7: Code tracing – recursion. What is the output?

```
def mystery(n, a, b, c):
    if n == 0:
        return

    mystery(n - 1, b, c, a)
    print(a, "->", c)
    mystery(n - 1, c, a, b)


mystery(3, "A", "B", "C")
```

Solution:

C -> B
B -> A
A -> C
A -> C
A -> C
C -> B
B -> A

Question 8: Code tracing: Add Two Numbers (Linked List Representation)

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None


def add_lists(l1, l2, carry=0):

    if not l1 and not l2 and carry == 0:
        return None


    val1 = l1.val if l1 else 0
    val2 = l2.val if l2 else 0


    total = val1 + val2 + carry
    new_val = total % 10
    new_carry = total // 10


    node = Node(new_val)


    next1 = l1.next if l1 else None
    next2 = l2.next if l2 else None


    node.next = add_lists(next1, next2, new_carry)

    return node


def print_list(head):

    while head:
        print(head.val, end=" -> ")
        head = head.next
    print("None")
```

Example Usage

```
# List 1: 2 -> 4 -> 3 (represents 342)
```

```
a1, a2, a3 = Node(2), Node(4), Node(3)
a1.next, a2.next = a2, a3

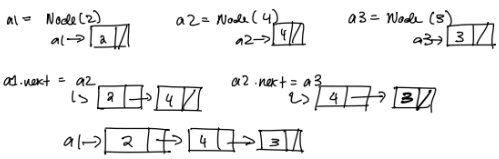
# List 2: 5 -> 6 -> 4 (represents 465)
b1, b2, b3 = Node(5), Node(6), Node(4)
b1.next, b2.next = b2, b3

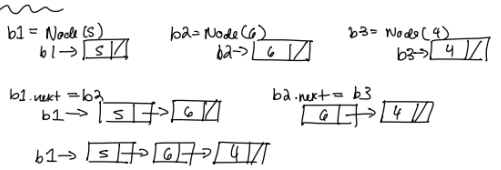
# Add them
result = add_lists(a1, b1)
print_list(result)
```

Solution:

7 -> 0 -> 8 -> None

Q 8: Code Tracing : Add Two Numbers (Linked List Representation)





Add them

result = add_lists(a1, b1)

(1) add_lists(a1, b1, carry=0):

l1=a1 , l2=b1, carry=0

doesn't return None (l1 == True and l2 == True)

val1 = 2
val2 = 5

total = 2 + 5 + 0 = 7
new_val = 7 % 10 = 7
new_carry = 7 // 10 = 0

node = Node(7) -> 7

next1 = 4
next2 = 6

node.next = add_lists(4, 6, 0)

(2) add_lists(4, 6, 0):

doesn't return None

val1 = 4
val2 = 6

total = 4 + 6 + 0 = 10
new_val = 10 % 10 = 0
new_carry = 10 // 10 = 1

node = Node(0) -> 0

next1 = 3
next2 = 4

node.next = add_lists(3, 4, 1)

(3) add_lists(3, 4, 1):

doesn't return None

val1 = 3
val2 = 4

total = 3 + 4 + 1 = 8
new_val = 8 % 10 = 8
new_carry = 8 // 10 = 0

node = Node(8) -> 8

next1 = None
next2 = None

node.next = add_lists(None, None, 0)

add_lists(None, None, 0):

l1 == False and l2 == False and carry == 0 :

return None

return 8

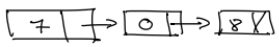
list : 8 -> None

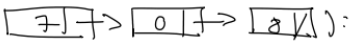
return 0

list : 0 -> 8

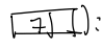
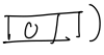
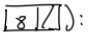
return 7

list : 7 -> 0 -> 8

result = 

print_list ():

Output:
7 -> 0 -> 8 -> None

1. while ():
   print (7, ->)
   head = 
2. while ():
   print (0, ->)
   head = 
3. while ():
   print (8, ->)
   head = None
4. head != True (while loop ends)

print (None)

Question 9: Code Tracing: Double ended queue: Round-Robin Processing

```
from collections import deque

def round_robin(tasks, quantum):

    queue = deque()
    elapsed = 0
    index = 0
    completed = []

    while index < len(tasks) or queue:

        if not queue and index < len(tasks):
            elapsed = max(elapsed, tasks[index][1])

        while index < len(tasks) and tasks[index][1] <= elapsed:
            name, arrival, remaining = tasks[index]
            queue.append((name, remaining))
            index += 1

        name, remaining = queue.popleft()

        used = min(quantum, remaining)
        elapsed += used
        remaining -= used

        while index < len(tasks) and tasks[index][1] <= elapsed:
            name_new, arrival, remaining_new = tasks[index]
            queue.append((name_new, remaining_new))
            index += 1

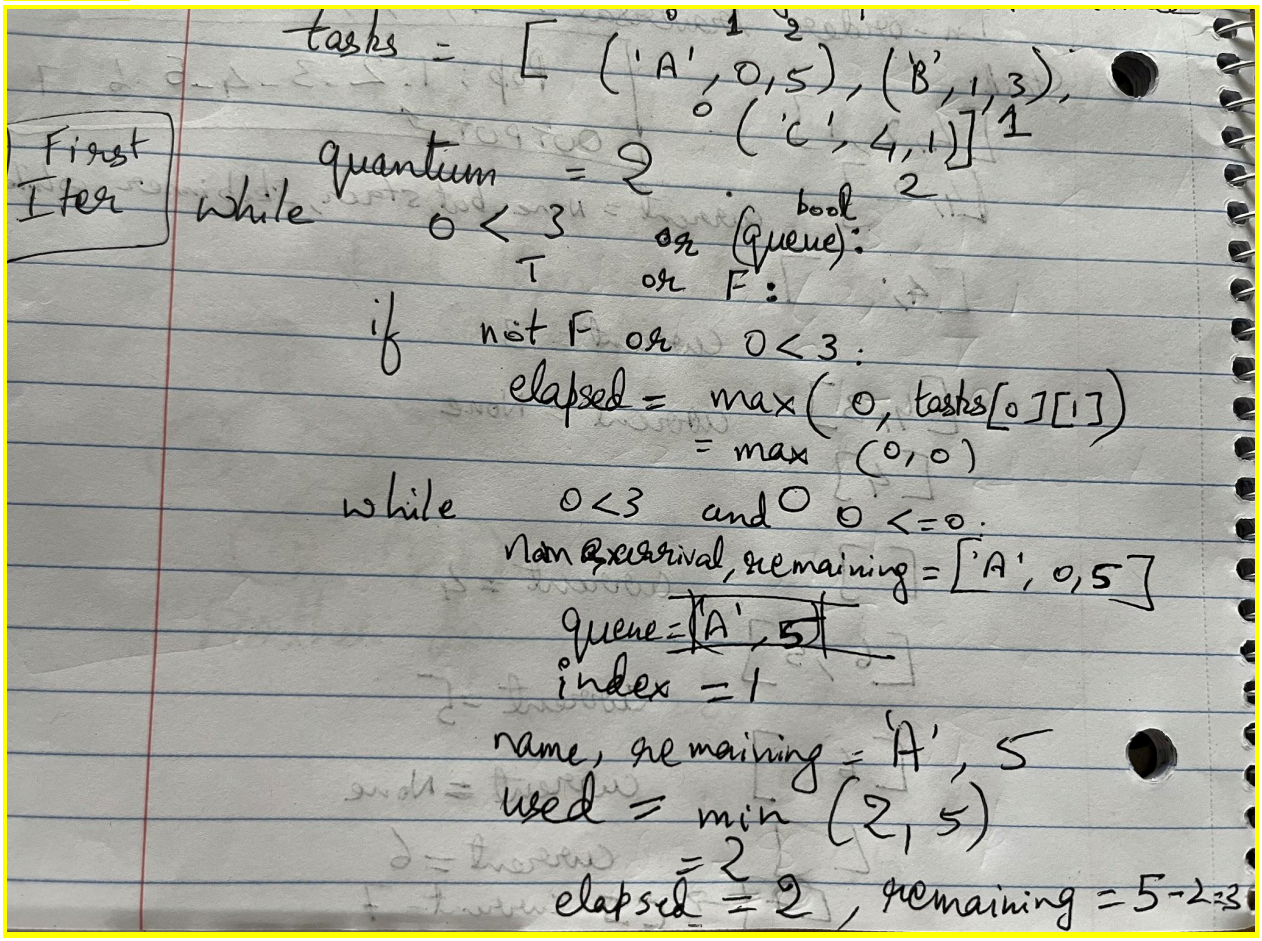
        if remaining > 0:
            queue.append((name, remaining))
        else:
            completed.append((name, elapsed))

    return completed
```

```
tasks = [
    ("A", 0, 5),
    ("B", 1, 3),
    ("C", 4, 1)
]

print(round_robin(tasks, 2))
```

Solution:



while 0 < 3 and tasks[1][1] <= 2:
0 < 3 and 1 ≤ 2 ? ✓
T T

name, arrival, remaining
'B' 1 5

Queue: | ('B', 3) |
index = 2

if 2 > 0 : ✓
T

Queue: | ('B', 3) | ('A', 3) |
bool

Second Iter

while 2 < 3 or (Queue):
F or not(F) = F or T = T

if not T and 2 < 3
F and T = F ×

while 2 < 2 and tasks[2][1]
F and ×
won't enter

name, remaining = 'B', 3
used = min(2, 3) = 2
elapsed = 2+2
remaining = 3-2 = 1

Queue: | ('A', 3) |

while 2 < 2 and
F and × won't enter

if 1 > 0 :

Queue | ('A', 3) | ('B', 1) |

Third Iter:

if not queue and 2 < 2 ↵ won't enter

while 2 < 2 and tasks[2][1] < : ↵ won't enter

name, remaining = 'A', 3

used = min(2, 3) = 2

elapsed = 4 + 2 = 6

remaining = 3 − 2 = 1

Queue ~~[('B', 3)]~~

while 2 < 2 and F ↵ won't enter

if 1 > 0: ✓

Queue [('B', 3), ('A', 1)]

Fourth Iter:

if not queue and 2 < 2 ↵ won't enter

while 2 < 2 and tasks[2][1] < ↵ won't enter

name, remaining = 'B', 1

used = min(2, 1) = 1

elapsed = 6 + 1 = 7

remaining = 1 − 1 = 0

Queue [('A', 1)]

while 2 < 2 and ↵ won't enter

completed = [('B', 7)]

Fifth Iter

if not queue and 2 < 2 ↵ ✗

while ↵ ✗

name, remaining = 'A', 1

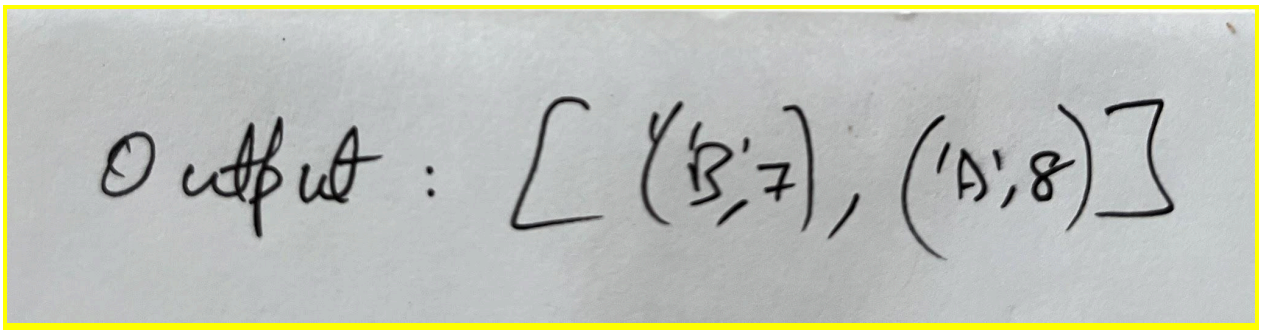
used = 1

elapsed = 8

remaining = 0

Queue = ——

completed = [('B', 7), ('A', 8)]




```
[('B', 7), ('A', 8)]
```

Question 10: Programming- Determine Whether Two Binary Trees Are Identical Using Stacks

Problem description

Write a function `identical_trees_stack(root1, root2)` that returns `True` if two binary trees are identical and `False` otherwise.

Two binary trees are **identical** when they have:

- The same structure: children exist in the same positions.
- The same keys at corresponding positions.

Each node has the attributes `key`, `lChild`, and `rChild`. Two empty trees are identical. If only one tree is empty, they are not identical.

Do not use recursion, tuples, or modify either tree.

Example: Two trees with root `4`, left child `2`, and right child `6` are identical. If one tree instead has `2` as its right child and `6` as its left child, they are not identical.

Hint:

1. 1. Use two stacks, one for each tree, beginning with their respective roots.
2. 2. Use a loop to compare saved positions. Keep the stacks synchronized by adding and removing corresponding entries together.
3. 3. After retrieving an entry from each stack, distinguish three cases: both are None, only one is None, or both contain nodes. Decide what each case means before accessing any attributes.
4. 4. When both nodes exist, compare their keys. A mismatch means the trees are not identical.
5. 5. Save corresponding left and right children in the same order. Include None entries so missing children can be compared.
6. 6. Consider what to return if every comparison finishes without a mismatch.

Solution:

```
def identical_trees_stack(root1, root2):
    stack1 = [root1]
    stack2 = [root2]

    while stack1:
        current1 = stack1.pop()
```

```
        current2 = stack2.pop()

        # Both positions are empty, so they match.
        if current1 is None and current2 is None:
            continue

        # Only one position is empty, so structures differ.
        if current1 is None or current2 is None:
            return False

        # Both nodes exist, so compare their keys.
        if current1.key != current2.key:
            return False

        # Save corresponding children, including missing ones.
        stack1.append(current1.rChild)
        stack2.append(current2.rChild)

        stack1.append(current1.lChild)
        stack2.append(current2.lChild)

    return True
```

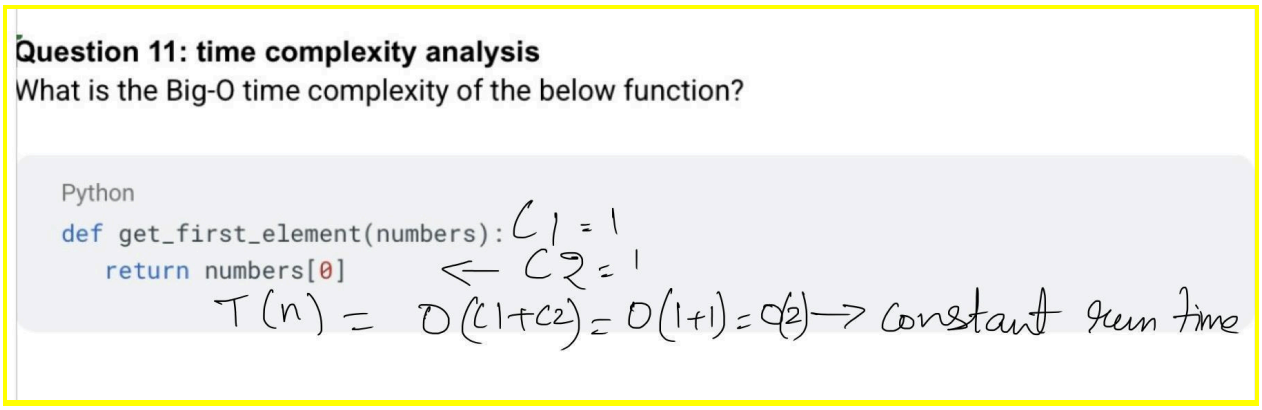
Question 11: Conceptual - Analysis/Complexity

Question 11a: time complexity analysis

What is the Big-O time complexity of the below function?

```
def get_first_element(numbers):
   return numbers[0]
```

Solution:



Question 11b: conceptual question (Linked List Structure)

Draw the variables, references, and objects that exist after the following code executes. Label the `prev` and `next` references in your node drawings. Identify the data structure here and briefly defend your answer.

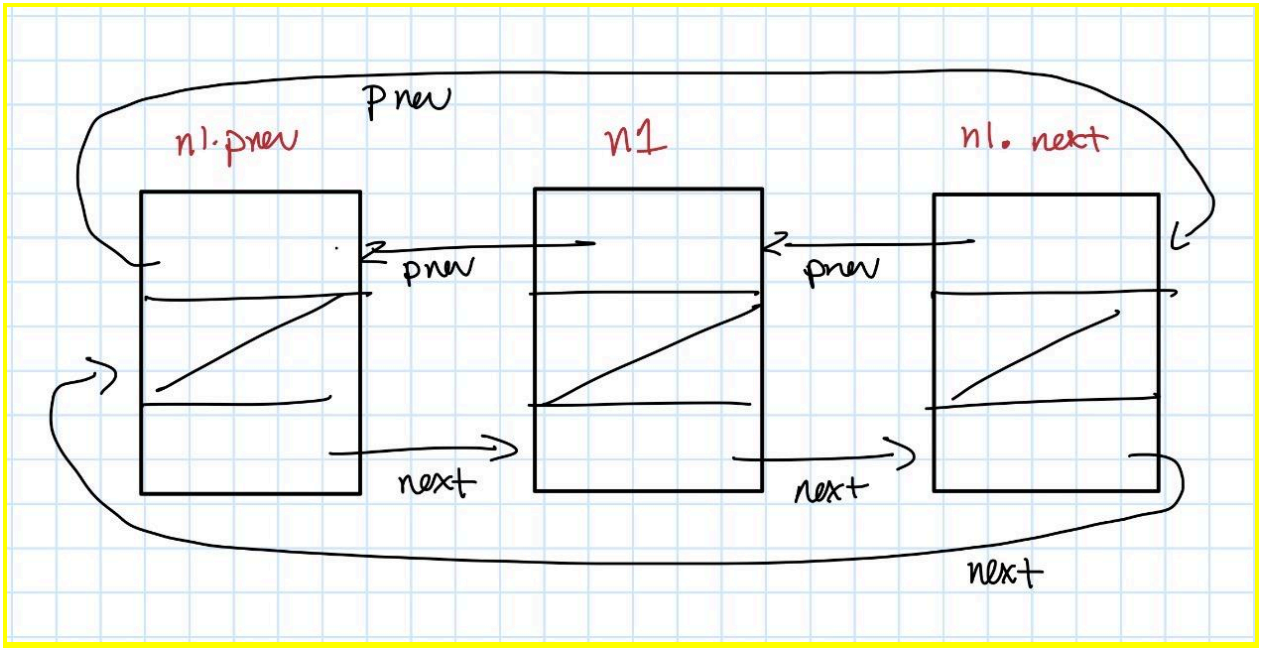
```
class Buddy:
    def __init__(self):
        self.data = None
        self.next = None
        self.prev = None

n1 = Buddy()
n1.next = Buddy()
n1.prev = Buddy()

n1.next.next = n1.prev
n1.next.prev = n1

n1.prev.next = n1
n1.prev.prev = n1.next
```

Solution:

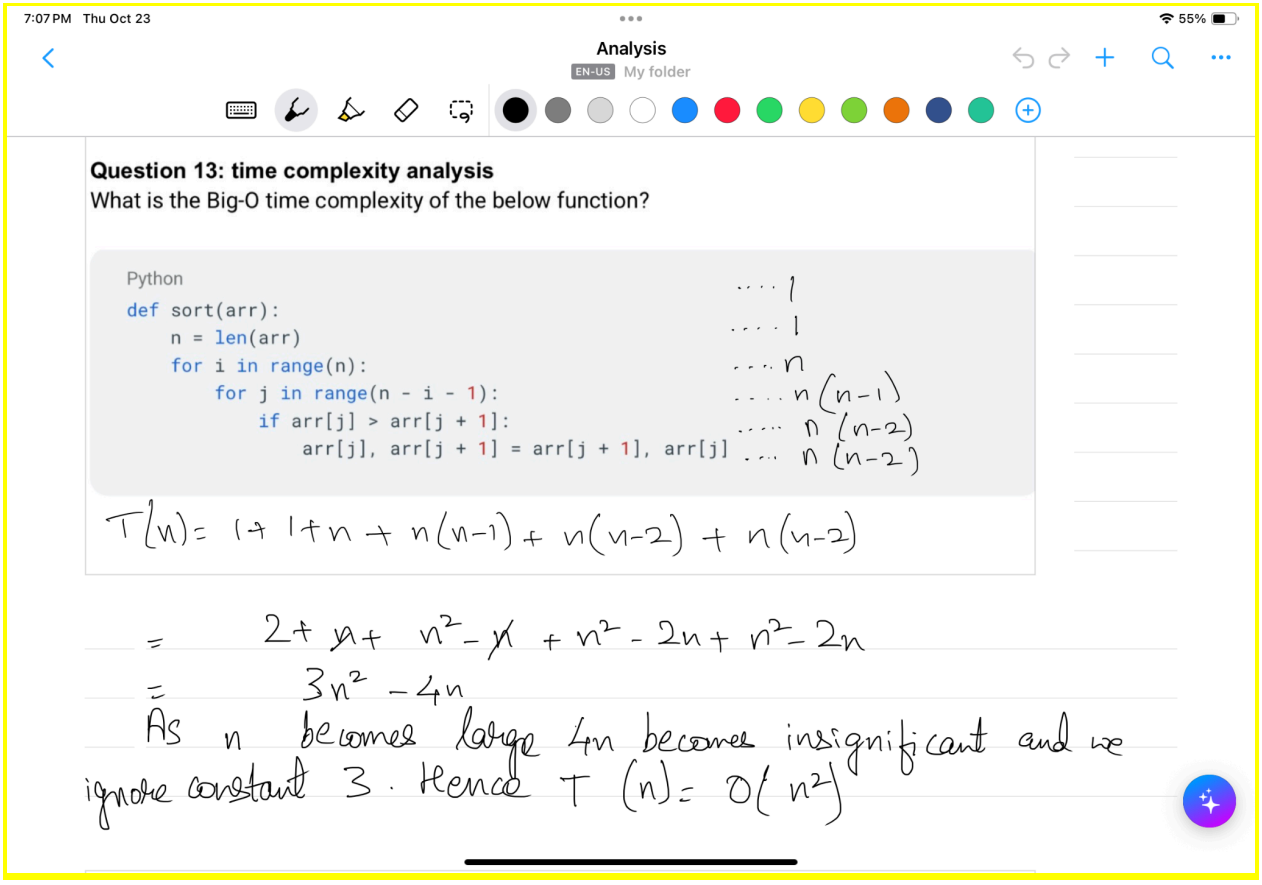


Question 11c: time complexity analysis

What is the Big-O time complexity of the below function?

```
def sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

Solution:



Question 11d: time complexity analysis

```
n = 5  # example value, you can change

for i in range(0, n):
    count = 0
    x = i * n
    print("i, x:", i, x)

    for j in range(0, x):
        count += 1
        print("count:", count)
```

Solution:

$i \leq (n-1)$

$x \leq (n-1) \cdot n$

Outside loop runs n times.

Nested loop runs $n \cdot x$ or $n \cdot (n \cdot (n-1))$ times.

$O(n^3)$

Question 12: Code tracing: Permutations

What is the output? Write out all steps of the memory trace.

```
def permutation(lst, start, end):
      if start == end:
          print(lst)
      else:
          for i in range(start, end + 1):
              lst[start], lst[i] = lst[i], lst[start]     # choose
              permutation(lst, start + 1, end)            # explore
              lst[start], lst[i] = lst[i], lst[start]     # UNCHOOSE

  permutation(['B','E','V'], 0, 2)
```

Solution (starts on next page):

Code tracing permutation

permutation(['B','E','V'], 0, 2)
start end
if 0 == 2? X 0 1 2
else: i in range(0,3) = 0, 1, 2
Outermost loop - iteration i = 0
start = 0; end = 2

i : 0

choose
lst[0], lst[0] = lst[0], lst[0] ← list unchanged by swaps
exploration
permutation(lst, 1, 2)
start = 1 end = 2

if 1 == 2? X
else:
i in range(1,3): 1, 2

i = 1

choose
list unchanged → lst[1], lst[1] = lst[1], lst[1]
exploration
permutation(lst, 2, 2)
start end

if 2 == 2?
print!
return to caller

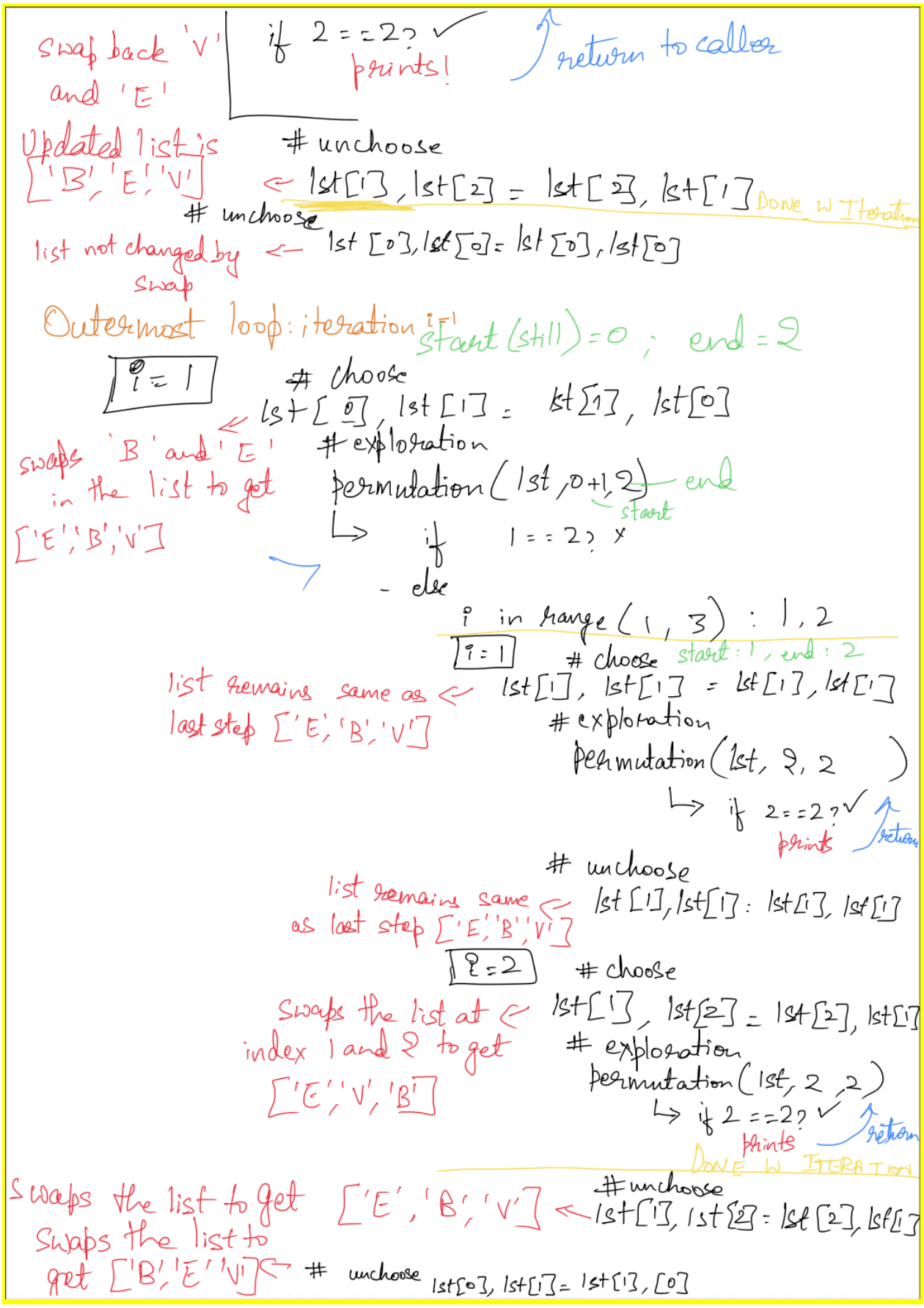
unchoose
list unchanged → lst[1], lst[1] = lst[1], lst[1]

i = 2

choose
Swaps 'E' and 'V'. New list is
lst[1], lst[2] = lst[2], lst[1]
exploration
permutation(lst, 2, 2) →

['B', 'V', 'E']

Output:
['B', 'E', 'V']
['B', 'V', 'E']
['E', 'B', 'V']
['E', 'V', 'B']
['V', 'E', 'B']
['V', 'B', 'E']


swap back 'V' and 'E'
if 2==2? ✓ prints!
return to caller
Updated list is ['B','E','V']
unchoose
lst[1], lst[2] = lst[2], lst[1]
DONE W ITERATION
unchoose
list not changed by swap
lst[0], lst[0] = lst[0], lst[0]
Outermost loop: iteration i=1
start (still) = 0 ; end = 2
i = 1
choose
lst[0], lst[1] = lst[1], lst[0]
swaps 'B' and 'E' in the list to get ['E','B','V']
exploration
permutation(lst, 0+1, 2)
start
end
if 1==2? ✗
else
i in range(1, 3) : 1, 2
i = 1
choose start: 1, end: 2
list remains same as last step ['E','B','V']
lst[1], lst[1] = lst[1], lst[1]
exploration
permutation(lst, 2, 2)
if 2==2? ✓
prints
return
unchoose
list remains same as last step ['E','B','V']
lst[1], lst[1] = lst[1], lst[1]
i = 2
choose
Swaps the list at index 1 and 2 to get ['E','V','B']
lst[1], lst[2] = lst[2], lst[1]
exploration
permutation(lst, 2, 2)
if 2==2? ✓
prints
return
DONE W ITERATION
unchoose
Swaps the list to get ['E','B','V']
lst[1], lst[2] = lst[2], lst[1]
Swaps the list to get ['B','E','V']
unchoose
lst[0], lst[1] = lst[1], [0]

Outermost loop: iteration 2

i = 2 start: 0 end: 2

choose

lst[0], lst[2] = lst[2], lst[0]

swaps the list to get ['V', 'E', 'B']

exploration

permutation(lst, 1, 2) (start, end)

↳ if 1 == 2? ×

else

i in range(1, 3): 1, 2

i = 1 # choose

lst[1], lst[1] = lst[1], lst[1]

swaps keep list the same ['V', 'E', 'B']

exploration

permutation(lst, 2, 2)

↳ if 2 == 2? ✓ prints

returns

unchoose

lst[1], lst[1] = lst[1], lst[1]

swaps keeps list the same ['V', 'E', 'B']

i = 2

choose

lst[1], lst[2] = lst[2], lst[1]

swaps list to ['V', 'B', 'E']

exploration

permutation(lst, 2, 2)

↳ if 2 == 2? × prints!

returns

DONE W ITERATION

unchoose

lst[1], lst[2] = lst[2], lst[1]

swaps list to ['V', 'E', 'B']

unchoose

lst[0], lst[2] = lst[2], lst[0]

swaps to ORIG list ['B', 'E', 'V']

DONE! We generated all permutations of ['B', 'E', 'V']